

Kontakt

Materiały

Przetwarzanie Obrazów

Systemy Multimedialne

Interaktywne Systemy
MultimedialnePrzetwarzanie i analiza
danychFAQ - najczęściej zadawane
pytania[Home](#) > [Systemy Multimedialne](#) > Kompresja video

Kompresja video

W ramach dzisiejszych zajęć połączymy wiedzę z naszych zajęć dotyczących kompresji dźwięku i obrazu i postaramy się opracować własny program kompresujący wideo. Jako część instrukcji został dołączony fragment programu, który będzie rozwijany w ramach zajęć.

Co przetestować w ramach sprawozdania

Sprawozdanie dzielimy na dwie części:

1. Badanie jakości dla różnych parametrów *bez użycia RLE lub ByteRun (0.6 pkt)*:

W tej części należy sprawdzić, jak poszczególne parametry wpływają na jakość skompresowanej klatki. Proszę porównywać wizualnie różnice w przestrzeni **RGB**, sprawozdania oceniające różnice tylko w przestrzeni **YCbCr** nie będą oceniane. Znaleźć najbardziej obiecujące zestawienie parametrów, które według was będzie dawało największe szanse na dalszą lepszą kompresję w drugiej części zadania. Przetestować:

- stopień redukcji chrominancji na 6 sposobów: **4:4:4**, **4:2:2**, **4:4:0**, **4:2:0**, **4:1:1**, **4:1:0**
- stopień kompresji informacji o różnicy względem klatki kluczowej (dzielenie przez 1,2,4,...)

Proszę pamiętać, żeby załączane do sprawozdania pliki były w jakości, pozwalającej potwierdzić to, co piszecie w obserwacjach. tu prezentujecie wyniki działania rozszerzonej przez was funkcji **plotDiffrence** prezentujące różnice w jakości względem klatki nie-kluczowej, **nie wykresy linowe, które generują się na końcu działania programu**. Samą funkcję można jeszcze bardziej rozszerzyć, żeby była bardziej podobna do tej z instrukcji o JPEG i pokazywała różnice na większej ilości płaszczyzn (**YCbCr**), ale pamiętajcie, że najważniejsze obserwacje i tak będziecie obserwować na obrazach w RGB. Dołączony do zadania skrypt generuje wykresy, dla wskazanego ROI (ang. Region of Interest). Proszę pamiętać, żeby ustawić ROI na obszar, który ulega zmianie pomiędzy klatkami, a nie domyślny lewy górny róg klatki. I sugeruję obserwować klatki jak najdalej odsunięte od klatek kluczowych, czyli celować w porównania w klatkach poprzedzających klatki kluczowe np. jeżeli mamy co 4 klatkę kluczową (0, 4, 8, ...) to do analizy wybierzmy klatkę 31.

2. Badanie skuteczności kompresji z *użyciem RLE lub ByteRun (0.4 pkt)*:

- Tutaj proszę wykorzystać najlepsze według was ustawienia z poprzedniej części zadania i wybrać **jeden** film do badań wstępnych. Algorytmy będą działały dość wolno, dlatego sugeruję wykonywać testy na kilku klatkach kluczowych, czyli maksymalnie 15-20 klatkach. Proszę wyraźnie zaznaczyć jaką metodę kompresji bezstratnej wykorzystujecie. Najlepiej dodajcie tę informację w tytule waszych wykresów liniowych (to te, które generują się na końcu programu). Proszę tu odpowiedzieć sobie na kilka pytań:
 - Jak zmiana odległości pomiędzy klatkami kluczowymi wpływa na kompresję? Jaka odległość jest najrozsądniejsza w przypadku waszego filmu? Proszę to przetestować na minimum 6 odległościach z zakresu $< 2, 20 >$ klatek.
 - Czy na podstawie waszych wyników opłaca się kompresować Luminancję, poprzez użycie kompresji strumieniowej? Jeżeli tak to dla jakich klatek (kluczowych, niekluczowych)?
- Po przygotowaniu powyższego badania i zapianiu ich w sprawozdaniu wszystkich ustawień dla waszego bazowego filmu, a następnie uruchomić i sprawdzić te same ustawienia na innym filmie. Czy te wnioski są dalej dla niego zgodne? Jeżeli nie to dlaczego tak jest?

Do oddania

- kod źródłowy (jeden plik **.py**)
- sprawozdanie z obserwacjami i wynikami (format **PDF**)

Program startowy

Dołączony do instrukcji **program startowy** składa się z kilku sekcji: Konfiguracji, Sekcji na własne funkcje służące do kompresji i dekompresji oraz z głównej pętli programu. Możecie spokojnie modyfikować każdą z części programu. Podział ten ma służyć głównie porządkowaniu struktury programu. **UWAGA** Użyty przykładowy schemat rysowania klatek kluczowych jest specjalnie zostawiony w niewłaściwej przestrzeni koloru, należy to poprawić w ramach prac nad programem, aby działał zgodnie z naszymi założeniami.

Sekcja dotycząca konfiguracji zawiera zmienne wykorzystywane w głównej pętli programu. Znajdziemy tu takie parametry jak:

- Katalog i nazwa pliku, (osoby korzystające z Linuxa lub MacOSa muszą tu zmienić ścieżki)
- ilość klatek filmu do odtworzenia,
- informację, co która klatka ma być klatką kluczową,
- dla których klatek w pętli głównej wyrysować wizualizację,
- dla których klatek zatrzymać automatycznie film (naciśnij dowolny klawisz, żeby od pauzować),
- rodzaj redukcji chrominancji,
- dzielnik dla kodowania różnic,
- czy program ma na bieżąco wyświetlać klatki, czy przeprowadzić kompresję w tle.
- ROI, czyli lista fragmentów, dla których będą generować się automatyczne wykresy.

Sekcja z funkcjami aktualnie tylko przepisuje naszą macierz wejściową do struktury po kompresji i na odwrót.

Główna pętla programu ma kilka najważniejszych sekcji. Na początku pobieramy nową klatkę i konwertujemy ją do przestrzeni **YCbCr**. Następnie w zależności czy mamy teraz nową klatkę kluczową lub nie wykonujemy operacje przeznaczone dla klatki kluczowej, lub dla klatki zwykłej. Potem mamy sekcję szacowania kompresji, po niej następuje sekcja automatycznego generowania wykresów. Program posiada dwie możliwości zatrzymania — automatyczną ustawianą dla konkretnych klatek na początku lub ręczną po naciśnięciu klawisza **p**. Obie trzeba od pauzować, naciskając dowolny przycisk na klawiaturze. Dodatkowo naciśnięcie klawisza **q** powoduje przerwanie i zamknięcie programu.

Kompresja

Kontakt

Materiały

Przetwarzanie Obrazów

Systemy Multimedialne

Interaktywne Systemy
MultimedialnePrzetwarzanie i analiza
danychFAQ - najczęściej zadawane
pytania

W ramach kompresji będziemy kompresować dwa aspekty sygnału: Zamiast całych klatek będziemy zapisywać różnice pomiędzy nimi a klatkami kluczowymi. Drugim aspektem będzie redukcja chrominancji, która była już poruszana na laboratoriach z kompresji JPEG. W ramach dzisiejszej instrukcji rozszerzymy naszą funkcję redukującą chrominancję o 4 kolejne możliwe podejścia. Czyli oprócz implementowanych ostatnio: **4:4:4** oraz **4:2:2**, powinna ona również obsługiwać **4:4:0**, **4:2:0**, **4:1:1** oraz **4:1:0**.

Kompresja video — różnica od klatki kluczowej

Kodowanie różnic było już omawiane przy okazji algorytmu DPCM w ramach kompresji stratnej dźwięku. Wtedy kodowaliśmy różnice względem sygnału estymowanego na wyjściu, dużym minusem tego podejścia jest niemożliwość swobodnego przewijania sygnału, ponieważ żeby odtworzyć wartość w konkretnym czasie musimy zrekonstruować cały sygnał. Dla sygnału audio zwykle nie jest to dla nas problem, ponieważ nie zajmuje on dużo miejsca w pamięci, jednak w przypadku sygnału wideo jest już to niestety dość problematyczne, ponieważ danych w jednej chwili jest dużo więcej. Rozwiązanie tego problemu są klatki kluczowe. Klatka kluczowa to klatka wzorcowa, do której są porównywane i na jej podstawie odtwarzane następujące po niej klatki filmu. Klatki kluczowe mogą być oddalone od siebie o stałą odległość, jak również mogą być rozmieszczone w sposób nierównomierny w zależności od kompresowanego materiału np. zmiany ujęcia. Klatki kluczowe mogą również być poddawane kompresji zarówno stratnej (redukcja chrominancji), jak i bezstratnej (RLE).

Kodowanie różnic to zwykła operacja odejmowania bieżących klatek od klatek źródłowych. Operację tą wykonujemy osobno dla każdej warstwy już po ewentualnej redukcji chrominancji. Nasze klatki mają wartości z zakresu $< 0, 255 >$, więc nasze różnice są w zakresie $< -255, 255 >$, dlatego można również sprawdzić, czy daje coś kompresja tego zakresu *podzielenie* go przez 2 przed kompresją i *pomnożenie* razy 2 przy dekompresji co dałoby nam możliwość zapisania go na 8 bitach (lub przez 4 dla jeszcze mniejszej redukcji).

UWAGA, Jeżeli będą się pojawiały 'duchy' w odtwarzanym obrazie to operacje odejmowania wykonujecie w złej kolejności. Polecam odjąć klatkę kluczową od klatki bieżącej (**Frame-Key**), zamiast klatkę bieżącą od klatki kluczowej (**Key-Frame**).

Przykład

Zakładamy, że mamy dwie klatki I_k - klatka kluczowa oraz I_n - klatka kodowana:

$$I_k = \begin{bmatrix} 20 & 34 & 58 \\ 18 & 25 & 33 \\ 230 & 28 & 0 \end{bmatrix} \quad I_n = \begin{bmatrix} 23 & 32 & 59 \\ 20 & 25 & 28 \\ 30 & 15 & 250 \end{bmatrix}$$

Teraz musimy wyliczyć różnicę pomiędzy naszymi klatkami R :

$$R = I_n - I_k = \begin{bmatrix} 23 & 32 & 59 \\ 20 & 25 & 28 \\ 30 & 15 & 250 \end{bmatrix} - \begin{bmatrix} 20 & 34 & 58 \\ 18 & 25 & 33 \\ 230 & 28 & 0 \end{bmatrix} = \begin{bmatrix} 3 & -2 & -1 \\ 2 & 0 & -5 \\ -200 & -13 & 250 \end{bmatrix}$$

Jak widać nasze R potrzebuje co najmniej jednego bitu więcej niż przestrzeń wartości dla jednej klatki, czyli do zapisu różnicy **uint8** potrzebujemy w teorii **int9**. Dlatego możemy również podzielić różnicę przez wielokrotność liczby 2 (4, 8, ...), wtedy będziemy mieli możliwość zapisania danych na mniejszej liczbie bitów. Przygotujemy sobie 2 dodatkowe zmienne R : R_2 - podzielone przez 2 oraz R_4 - podzielone przez 4

$$R_2 = \left\| \begin{bmatrix} 3 & -2 & -1 \\ 2 & 0 & -5 \\ -200 & -13 & 250 \end{bmatrix} * \frac{1}{2} \right\| = \begin{bmatrix} 2 & -1 & 1 \\ 1 & 0 & -3 \\ -100 & -7 & 125 \end{bmatrix} \quad R_4 = \left\| \begin{bmatrix} 3 & -2 & -1 \\ 2 & 0 & -5 \\ -200 & -13 & 250 \end{bmatrix} * \frac{1}{4} \right\| = \begin{bmatrix} 1 & -1 & 0 \\ 1 & 0 & -1 \\ -50 & -3 & 63 \end{bmatrix}$$

Jak widać duże wartości są znacząco mniejsze teraz spróbujemy odtworzyć nasze klatki O na podstawie różnic:

$$O = I_k + R = \begin{bmatrix} 20 & 34 & 58 \\ 18 & 25 & 33 \\ 230 & 28 & 0 \end{bmatrix} + \begin{bmatrix} 3 & -2 & -1 \\ 2 & 0 & -5 \\ -200 & -13 & 250 \end{bmatrix} = \begin{bmatrix} 23 & 32 & 59 \\ 20 & 25 & 28 \\ 30 & 15 & 250 \end{bmatrix}$$

$$O = I_k + 2 * R_2 = \begin{bmatrix} 20 & 34 & 58 \\ 18 & 25 & 33 \\ 230 & 28 & 0 \end{bmatrix} + 2 * \begin{bmatrix} 2 & -1 & 1 \\ 1 & 0 & -3 \\ -100 & -7 & 125 \end{bmatrix} = \begin{bmatrix} 20 & 34 & 58 \\ 18 & 25 & 33 \\ 230 & 28 & 0 \end{bmatrix} + \begin{bmatrix} 4 & -2 & 2 \\ 2 & 0 & -6 \\ -200 & -14 & 250 \end{bmatrix} = \begin{bmatrix} 24 & 32 & 60 \\ 20 & 25 & 27 \\ 30 & 14 & 250 \end{bmatrix}$$

$$O = I_k + 4 * R_4 = \begin{bmatrix} 20 & 34 & 58 \\ 18 & 25 & 33 \\ 230 & 28 & 0 \end{bmatrix} + 4 * \begin{bmatrix} 1 & -1 & 0 \\ 1 & 0 & -1 \\ -50 & -3 & 63 \end{bmatrix} = \begin{bmatrix} 20 & 34 & 58 \\ 18 & 25 & 33 \\ 230 & 28 & 0 \end{bmatrix} + \begin{bmatrix} 4 & -4 & 0 \\ 4 & 0 & -4 \\ -200 & -12 & 252 \end{bmatrix} = \begin{bmatrix} 24 & 30 & 58 \\ 22 & 25 & 29 \\ 30 & 16 & 252 \end{bmatrix}$$

Odtworzone klatki nie różnią się w znaczący sposób między sobą wartościami.

Chroma subsampling

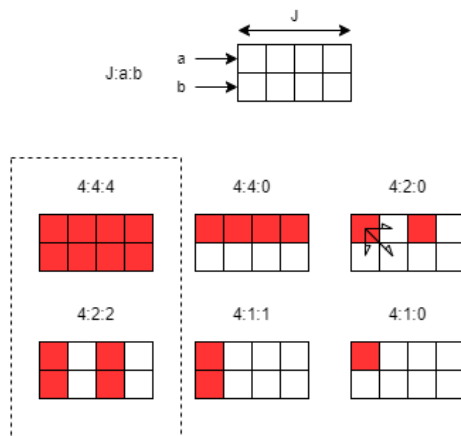
Redukcja chrominancji (ang. Chroma Subsampling) to jedna z operacji kompresji polegająca na zmniejszeniu informacji na temat koloru w naszym obrazie. Wykorzystujemy tutaj fakt, że ludzkie oko jest bardziej wrażliwe na zmianę jasności (Luminancja), niż koloru (Chrominancja). Subsampling zatem wykonujemy na warstwach **Cb** oraz **Cr** (dla każdej z nich osobno). Jego rodzaj opisywany jest przez 3 wartości opisane w ciągu **J:a:b** jak tak to pokazano na poniższym schemacie.

Kontakt

Materiały

Przetwarzanie Obrazów

Systemy Multimedialne

Interaktywne Systemy
MultimedialnePrzetwarzanie i analiza
danychFAQ - najczęściej zadawane
pytania

Pierwszy parametr J określa, jak dużo pikseli w poziomie będziemy analizować, zwykle jest to wartość 4. Kolejny parametr a określa, ile wartości z pierwszego analizowanego przez nas wiersza będziemy zapisywać, natomiast parametr b opisuje liczbę wartości w drugim wierszu. Na przykładzie graficznym zapisywane zostają wartości zaznaczone jako czerwone kwadraty. Jak widzimy w przypadku **4:4:4** wszystkie wartości zostają zapisane, więc nic nie ulega zmianie. W pozostałych przypadkach zawsze tracimy jakieś informacje. Działanie funkcji odwrotnej polega na odtworzeniu całej warstwy naszego obrazu na podstawie zapisanych przez nas wcześniej informacji. Jeżeli nie mamy dostępnej informacji o wartości piksela, to zastępujemy (w zależności gdzie się on znajduje) inną najbliższą wartością z tego lub poprzedniego wiersza. Przykładowo dla **4:4:0** będziemy uzupełniać drugi wiersz wartościami z pierwszego, dla **4:2:0** jedna wartość będzie propagowana na 4 piksele, dla **4:1:1** nowa wartość będzie zastępowała cały wiersz, a dla **4:1:0** nowa wartość będzie rozchodziła się dla wszystkich 8 pikseli. W ramach tych laboratoriów interesują nas dwie wersje **4:4:4** - czyli obraz bez zmian oraz **4:2:2** - czyli redukcja ilości pikseli o 50% w poziomie (każda wartość będzie odtwarzana za pomocą piksela na lewo od niego). Pamiętajcie, że sposób kompresji chrominancji naszego obrazu powinien również zostać zapisany wewnątrz naszej struktury, żeby dekodery również wiedział, w jaki sposób go odtworzyć. Więcej informacji na temat redukcji chrominancji można przeczytać w artykule na [Wikipedii](#) z innymi przykładami graficznymi.

Jak do tego podejść programistycznie?

Po pierwsze nie musimy procesu redukcji chrominancji przeprowadzać na poszczególnych blokach. Możemy na raz przetwarzać całą warstwę, czyli nie potrzebujemy żadnych pętli.

Druga podpowiedź, czyli w jaki sposób zdecydować jaką redukcję robić. Tutaj można spróbować coś automatyzować, ale w naszym przypadku najwygodniej napisać odpowiedni zestaw `if, elif, else`. Macie dzięki temu lepszą kontrolę nad tym, co działa, a co nie.

```
01. if ratio=="4:2:2":
02.     pass
03. elif ratio=="4:2:0":
04.     ## Nie dla Laboratoriów z JPEG
05.     pass
06. else: #default "4:4:4"
07.     pass
```

Trzecia podpowiedź, czyli jak przeprowadzić redukcję. Tu sprawa jest prosta, pakiet NumPy daje nam możliwość dowolnego adresowania danych, więc możemy dokonać redukcji poprzez wybranie elementów macierzy, które nas interesują. Przypominam, że adresowanie przebiega w kolejności `[wiersze, kolumny, warstwy]`. W każdym z adresów możemy podać generator określający, które elementy nas interesują. Adres podajemy w formacie `od:do:krok`, gdzie `od` i `do` możemy pominąć, jeżeli chcemy wektor od początku do końca zakresu, a krok domyślnie jest równy 1. Czyli jeżeli chcemy dostać wektor zawierający co 4-tą kolumnę i co drugi wiersz nasze zapytanie powinno wyglądać tak:

```
01. B=A[:, :2, ::4]
```

Ostatnia podpowiedź, czyli jak uzupełniać dane przy resampling'u. Tu można albo ręcznie uzupełniać wykorzystując adresowanie, albo wykorzystać ułatwienie w postaci funkcji `np.repeat`. Tylko należy pamiętać, żeby w przypadku kilku z przypadków trzeba wykonać wpiery powielanie dla wierszy, a następnie dla kolumn.

► Przykłady wizualne

Kompresja Strumieniowa

Dla każdej z naszych skompresowanych warstw koloru bez względu na to, czy będzie to klatka kluczowa, czy nie można spróbować je zredukować poprzez naszą kompresję bezstratną (RLE lub ByteRun) implementowaną na wcześniejszych zajęciach. Sama operacja kompresji jest dość czasochłonna, dlatego polecam robić to na niewielkiej ilości klatek, głównie w celu testów na najbardziej obiecujących parametrach.

Kontakt

Materialy

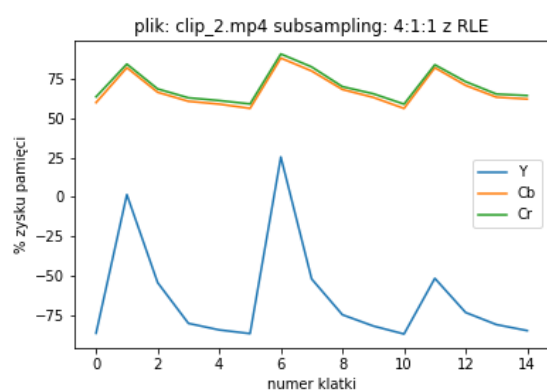
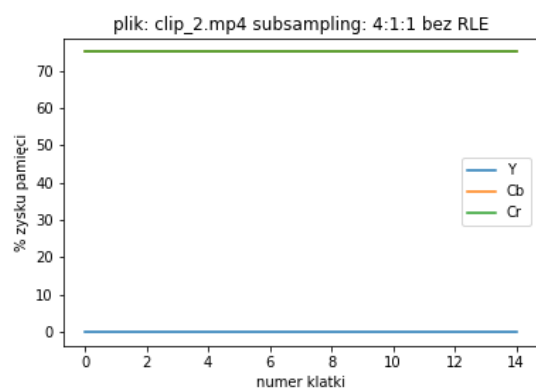
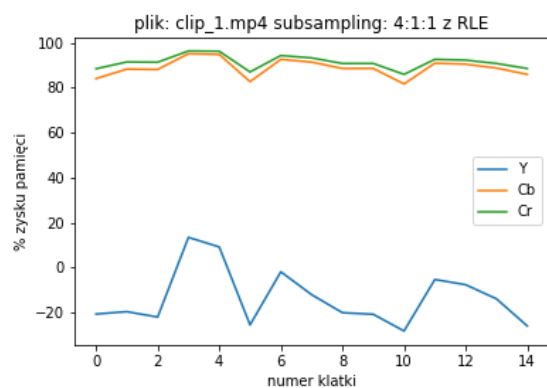
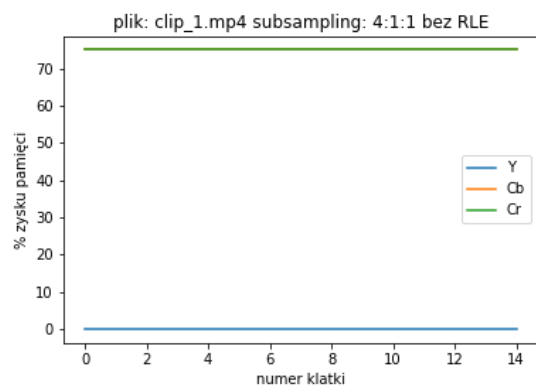
Przetwarzanie Obrazów

Systemy Multimedialne

Interaktywne Systemy
Multimedialne

Przetwarzanie i analiza
danych

FAQ - najczęściej zadawane
pytania



Kontakt

Materiały

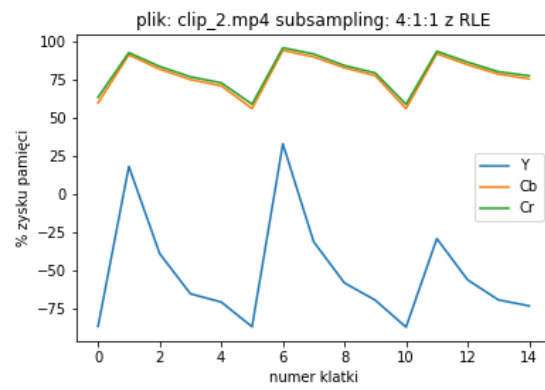
Przetwarzanie Obrazów

Systemy Multimedialne

Interaktywne Systemy
Multimedialne

Przetwarzanie i analiza
danych

FAQ - najczęściej zadawane
pytania



Strona tworzona przez M. Kramarczyka©
[Deklaracja dostępności](#)

